telcoin network

Telcoin Network

Technical Specification

Version 1.1 · 15 October 2026
Protocol v0.16.0-adiri
devs@telcoin.org

Table of Contents

1. Introduction **4**

2. Architecture overview **5**
- Consensus chain 5
- Execution chain 5
- Epoch chain 5
- How the three chains complement each other 6

3. Consensus layer **7**
- Certificate DAG 7
- Total ordering 7
- Equal voting weight 8
- Reputation scheduling 8
- Worker horizontal scaling 8

4. Execution layer **10**
- EVM block production 10
- Repurposed block header metadata 10
  - Repurposed Fields 11
  - Fixed / Unused Fields 12
- System calls 12
  - Pre-execution calls 13
  - Epoch-boundary calls 13
- Gas over-reservation penalty 13
- Smart contracts 14

5. Epoch boundaries **15**
- Reward distribution 15
- Slash application 15
- Committee shuffle 16
- Base fee adjustment 16
- Fork scheduling 17
- Fork gates 17

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

6. Validator lifecycle **19**
ConsensusNFT 19
Validator statuses 20
Lifecycle steps 20
Delegation 21

7. Node modes **22**
CvvActive 22
CvvInactive 22
Observer 22
Mode detection 23
Capabilities 23

8. Networking **25**
Transport 25
Identity and discovery 25
Gossipsub topics 26
Transaction forwarding 26
Request-response RPC 27
Peer scoring 28

9. Cryptographic primitives **29**
BLS12-381 signatures 29
Ed25519 signatures 30
Hashing 30
Intent-scoped signing 30
Signature verification 31

10. Trustless synchronization **32**
Step 1: Sync the epoch chain 32
Step 2: Sync consensus headers 33
Step 3: Re-execute batches 34
Security properties 34
Recovery after restart 34

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

1. Introduction

Telcoin Network is a proof-of-stake blockchain protocol that combines DAG-based BFT consensus (Narwhal-Bullshark) with EVM-compatible transaction execution. The protocol orders transactions through consensus before executing them against canonical EVM state under the Prague (Pectra) rules, which are active from genesis. Batches admit legacy, EIP-2930 and EIP-1559 transactions; blob transactions and EIP-7702 set-code transactions are not accepted. Validators are mobile network operators and their affiliates, vetted through Telcoin governance.

This document describes the protocol's architecture, consensus mechanics, execution rules, validator lifecycle, and networking layer. It assumes familiarity with proof-of-stake systems and the EVM.

- The reference implementation is open source at github.com/telcoin-association/telcoin-network under a dual Apache 2.0 and MIT license.

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

2. Architecture overview

The protocol separates responsibilities across three chains: the Consensus Chain orders transactions, the Execution Chain executes state changes, and the Epoch Chain records committee transitions. The ordering layer works over batch digests and has no knowledge of gas, account state, or virtual machine rules. Workers check each batch against a small set of execution rules before acknowledging it (the transactions decode, every transaction type is on the executable allowlist, the batch fits the gas limit, and it carries the epoch's base fee), but they never execute it. This modular design allows the execution environment to evolve independently.

Consensus chain

The Consensus Chain is a hash-linked sequence of blocks that record BFT ordering decisions. Each header carries a `parent_hash` linking to the previous header, a `sub_dag` holding the committed sub-DAG (the ordered headers of the committed certificates with the leader last, plus the reputation scores and commit timestamp listed under Total ordering), and a `number` denoting the header's height. A fourth field, `extra`, is reserved and always zero. The header's digest is a BLAKE3 hash over the parent hash, the sub-DAG digest, the block number (little-endian), and a 32-byte zero placeholder for the reserved field. Header numbers start at zero at genesis and increase by one with each header. This chain advances whenever the BFT protocol commits a new sub-DAG and is the authoritative record of transaction ordering for all nodes on the network.

Execution chain

The Execution Chain is a standard Ethereum-compatible chain of blocks. The protocol produces exactly one EVM block per committed batch. Blocks are only produced when batches contain transactions or when an epoch boundary requires closing. Block headers carry consensus metadata in repurposed Ethereum header fields described in detail below. This binding makes every execution block traceable to the consensus decision that ordered its contents.

Epoch chain

The Epoch Chain is a hash-linked sequence of EpochRecords. Each epoch generates exactly one record, signed by a super-quorum of its committee. Each record contains the epoch number, the current committee's BLS public keys, the next committee's BLS public keys, a parent hash linking to the previous epoch record, a record of the final execution state (the block number and hash of the last execution block in the epoch), and a record of the final consensus state (the block number and hash of the last ConsensusHeader in the epoch). A record's committee normally equals the previous

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

record's next committee. If governance ejects a validator during the epoch, the record carries the smaller committee, which is accepted only if every member was in the expected committee, at least four members remain, and at least two-thirds of the expected committee remain. The outgoing committee signs each EpochRecord with a super-quorum BLS aggregate signature: at least two-thirds of the committee's members, rounded down, plus one. This chain anchors committee transitions and provides a compact proof of finality across epoch boundaries.

How the three chains complement each other

Property	Consensus Chain	Execution Chain	Epoch Chain
Unit	ConsensusHeader	EVM block	EpochRecord
Linking	Parent hash of previous header	Parent hash of previous block	Parent hash of previous epoch record
Content	Committed sub-DAG: the ordered headers of the committed certificates, with the leader last, plus reputation scores and the commit timestamp	Transactions, receipts, state root	Committee keys, finality anchors
Verification	Hash chain back to a trusted header hash (an epoch record's final consensus hash or a consensus result signed by one-third plus one of the committee); certificate signatures are checked when certificates enter the DAG and are not stored with the header	Deterministic re-execution of the consensus output, anchored by each epoch record's final execution state	Super-quorum BLS aggregate signature
Advancement	Each BFT sub-DAG commit	Each committed batch and/or epoch boundary	Each epoch boundary
Purpose	Canonical transaction ordering	State transition and EVM compatibility	Committee rotation and cross-epoch finality

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

3. Consensus layer

Telcoin Network uses a DAG-based Byzantine Fault Tolerant consensus protocol composed of two logical sub-protocols: a data availability layer (Narwhal) that constructs a certificate DAG, and a total ordering layer (Bullshark) that derives a deterministic commit sequence from that DAG.

Certificate DAG

Each round, every committee member proposes a header that references the digests of batches its workers have had acknowledged and the certificates it holds from the previous round. Once a proposal collects a quorum of BLS signatures, it becomes a certificate. Each certificate references a set of prior certificates from the previous round, forming a directed acyclic graph (DAG). This structure provides two guarantees: data availability (every referenced batch is retrievable by honest validators) and causal ordering (each certificate encodes a partial history of the proposals that preceded it).

Total ordering

The ordering layer processes the certificate DAG to create a definitive, final order for all transaction data. The protocol operates in rounds. Every even round has a leader, chosen by a deterministic rotation through the committee that reputation scheduling can adjust (see Reputation scheduling). A leader is committed once at least $f+1$ certificates of the following round reference it, enough to guarantee that at least one honest validator saw it. Here f is the number of faulty members the committee tolerates: the largest whole number below one-third of its size. When this happens, the protocol automatically commits the leader's certificate and every previous uncommitted certificate linked to it.

Within a committed sub-DAG, certificates are ordered by round, and the leader comes last. Certificates of the same round are ordered by a hash of the epoch's seed chain value (see Committee shuffle) and the certificate digest, a value no proposer can predict when it builds its header, so no validator can choose where its batches land in the execution order.

Each committed sub-DAG contains:

- Headers: the headers of the committed certificates, ordered by round, each referencing transaction batches (signatures are checked before the commit and not kept)
- Leader: the last header, the anchor from which the sub-DAG is derived

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

- Reputation scores: each validator's running score for the current scheduling window
- Commit timestamp: the canonical time assigned to the commit

The protocol decouples data dissemination from transaction ordering, allowing both to advance at network speed.

Equal voting weight

Every member of the epoch's committee carries one equal vote in consensus, regardless of individual stake amount. This prevents large stakeholders from dominating block production or commit decisions. Staked validators outside the committee for that epoch do not vote. Each validator stakes the fixed amount that its stake version requires. Epoch rewards are shared in proportion to effective stake (that amount, reduced if slashed) multiplied by the number of committed leader turns the validator held in the epoch, while consensus power stays uniform across the committee.

Reputation scheduling

The protocol keeps a reputation score for each committee member over a window of 60 leader rounds (120 consensus rounds). Scores reset at the end of each window and at the start of each epoch. A validator gains a point each time its certificate in the round after a committed leader references that leader. These scores influence the leader scheduling algorithm. Leadership rotates evenly through the committee. At the end of each window, validators whose scores fall far below the best (roughly two standard deviations, and normally no more than a third of the committee) lose their leader turns for the next window. Each lost turn goes to a validator drawn deterministically from the top scorers. This mechanism encourages liveness and penalizes absent or slow validators without requiring explicit slashing at the consensus layer. At the end of each epoch, every validator whose leader turns were committed receives a share of the epoch's issuance (see Reward distribution).

Worker horizontal scaling

Each validator operates one or more workers that independently construct transaction batches. Workers are parallel lanes for data ingestion and must be uniform across all validator nodes. Each worker lane maintains its own base fee and processes transactions concurrently with other workers on the same validator. Each worker has

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

its own transaction pool and its own RPC endpoint, so a transaction joins the lane of the endpoint it was submitted to. Every batch produced by a worker carries the following metadata:

- Beneficiary: the validator's address, used for priority fee distribution
- Base fee: the worker's base fee for the epoch, the same for every validator's worker with that ID and changed only at an epoch boundary
- Epoch: the epoch the batch was built for; peers reject a batch from any other epoch
- Worker ID: identifies the originating worker lane
- Transactions: the ordered set of transactions in the batch

Scaling the number of workers is configurable through on-chain governance at each epoch boundary. The DAG and ordering layers are agnostic to the number of workers per validator and operate solely over certificates that reference batches regardless of origin. The design leaves room for workers dedicated to other execution environments, though every worker carries EVM transactions.

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

4. Execution layer

The execution layer processes committed sub-DAGs and maintains the EVM state trie. Sub-DAGs technically only contain batch digests. The sub-DAG batch digests are used to map batch data so the execution layer has full transaction bytes before execution. The protocol refers to sub-DAGs with restored batch data as "consensus output". Consensus output contains all ordered batches of transactions which are executed to apply deterministic state transitions, enforcing protocol rules through a combination of user transactions and system calls.

EVM block production

Each batch within a consensus output produces exactly one EVM block. Blocks are created sequentially in batch order, and each block's resulting state changes are committed to the EVM state trie.

When a consensus output contains no batches, block production depends on the epoch boundary. If the consensus output contains no batches and is the last committed sub-DAG for the epoch, the execution layer conducts closing epoch protocol transactions using system calls. Otherwise, no state changes occur.

Every EVM instance carries two protocol precompiles beside the standard Ethereum set: the TEL precompile, which manages issuance of the native token (governance-only timelocked mint, claim and burn, plus a public total-supply view), and a BLS12-381 signature verifier, which the `ConsensusRegistry` uses to check validators' proofs of possession (see BLS12-381 signatures). They sit at these addresses:

```
TEL precompile       0x00000000000000000000000000000000000007e1
BLS12-381 verifier   0x000000000000000000000000000000000000b151
```

Repurposed block header metadata

The protocol repurposes several Ethereum block header fields to carry consensus-layer metadata. The consensus metadata in `nonce`, `difficulty`, `ommers_hash`, `extra_data` and `withdrawals` does not affect state changes and is included to support indexing tools, block explorers, and other services that read block headers directly. The `mix_hash` field supplies the `PREVRANDAO` value and `parent_beacon_block_root` is written to the beacon roots contract, while `beneficiary`, `timestamp` and `base_fee_per_gas` keep their usual EVM roles.

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

Repurposed Fields

Field	Ethereum Meaning	TN Meaning
`nonce`	PoW mining nonce	Epoch and consensus round, packed as: `(epoch << 32) | round`
`difficulty`	Network difficulty	Worker ID and batch index, packed as: `(batch_index << 16) | worker_id`
`mix_hash`	PoW mix digest (`PREVRANDAO` after the Merge)	The block's `PREVRANDAO`: `keccak256("TN_PREVRANDAO_V1" || seed || consensus block number || batch index)`, where the seed is the epoch seed chain value as of this commit and the last two values are little-endian 64-bit integers. It is unique per block and resists payload grinding, but the committing leader can compute it before broadcasting, so contracts that need unbiasable randomness must not rely on it alone
`ommers_hash`	Uncle block hash	Digest of the consensus batch executed to produce this block. `B256::ZERO` if closing the epoch with zero batches
`parent_beacon_block_root`	Beacon chain parent root	Digest of the ConsensusHeader that committed the transactions
`extra_data`	Arbitrary miner data	On the block that closes an epoch, the 32-byte committee-shuffle seed (the epoch seed chain value as of the closing commit); empty bytes otherwise
`base_fee_per_gas`	Adjusts per block (EIP-1559)	Fixed for the entire epoch for each worker lane and adjusted at epoch boundaries, either by the EIP-1559 rule against the lane's gas target or to a governance-set static fee
`withdrawals`	Beacon chain withdrawals	Validator reward records at epoch boundaries, otherwise empty

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

Field	Ethereum Meaning	TN Meaning
`beneficiary`	Block proposer's fee recipient	Execution address carried in the batch by the validator that produced it, which receives the batch's priority fees. The empty block that closes an epoch names the committing leader's execution address
`timestamp`	Proposer-set block time	Commit time of the consensus sub-DAG in whole seconds, shared by every block from one consensus output. The `tn_getBlockTimestampMillis` RPC method returns the commit time in milliseconds

Fixed / Unused Fields

Field	Value	Notes
`requests_hash`	`EMPTY_REQUESTS_HASH`	EIP-7685 deposit requests not used
`excess_blob_gas`	`0`	Blob transactions are not supported
`blob_gas_used`	`0`	No blob transactions are executed
`gas_limit`	30,000,000	Maximum gas per batch; changes only at a protocol fork

System calls

System calls are protocol-level transactions that modify contract state without user initiation. They share the following properties:

Property	Value
Caller	`SYSTEM_ADDRESS` (`0xfffffffffffffffffffffffffffffffffffffffe`)
Gas limit	100,000,000 per call, outside the block gas limit
Gas price	0 (no fee charged)
Nonce check	Disabled

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

System calls do not appear in the block's transaction list, produce no receipts, and do not count toward the block's gas used. Their effects are visible as contract state, but events they emit are not included in receipts or returned by `eth_getLogs`.

Pre-execution calls

Two pre-execution system calls run before a block's transactions. The first runs once per consensus output, in that output's first block; the second runs in every block:

1. Consensus root storage (EIP-4788 style): Writes the ConsensusHeader digest to the beacon roots contract, making consensus-layer data accessible to smart contracts.
2. Block hash history (EIP-2935): Stores the parent block hash in the history contract, extending the range of historically accessible block hashes.

Epoch-boundary calls

Four additional system calls run in the last block of each epoch: `applyIncentives()`, `applySlashes()`, and `concludeEpoch()` on the `ConsensusRegistry`, then `setWorkerConfigsData()` on the `WorkerConfigs` contract. The last call records the next epoch's base fee for each worker lane priced by EIP-1559. See Epoch boundaries for more details.

Gas over-reservation penalty

Because consensus orders transactions before they execute, a batch reserves capacity by each transaction's declared gas limit rather than by the gas it ends up using. To keep that capacity from being claimed for free, execution charges a penalty on any transaction whose gas limit exceeds 210,000 and which spends less than 10 percent of that limit, whether it succeeds or reverts; system calls are exempt. The penalty is computed in gas units with deterministic integer arithmetic as `((10^8 - r)^2 × u) / 10^16`, where `r` is the gas spent before EVM refunds divided by the gas limit and scaled by `10^9`, and `u` is the gas limit less that gas spent, so it grows quadratically as usage falls below 10 percent and never exceeds the unused gas. A 21,000-gas transfer sent with a 420,000 gas limit forfeits 25 percent of its unused gas, and the same transfer sent with a 30,000,000 gas limit forfeits 98.6 percent. The penalty is taken from the sender's unused-gas refund at the transaction's effective gas price, so the sender never pays more than the gas limit times that price. Like the base fee, the penalty is not burned: it is credited to the chain's base-fee address for governance processing. The receipt does not show the charge, because `gasUsed` excludes it and no log marks it, so wallets and indexers recover it as the sender's balance change less the value sent and `gasUsed × effectiveGasPrice`.

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

Smart contracts

The protocol relies on three system contracts placed at fixed addresses in the genesis state. The `ConsensusRegistry` is built on `StakeManager`, a staking module it inherits, so the two share one address:

1. `ConsensusRegistry`: Manages the validator lifecycle including registration, committee membership transitions, epoch metadata storage, and reward distribution.
2. `StakeManager`: The staking module inside the `ConsensusRegistry`. It holds the stake versions, validator balances and delegations, and the ConsensusNFT (ERC-721) ledger: a non-transferable token that governance mints to each approved validator address and that the address must hold before it can stake. Supports delegation.
3. `WorkerConfigs`: Stores the number of workers and each worker's base-fee policy on-chain: a fee strategy (EIP-1559 or a static fee), its parameter (target gas or flat fee), and a data field. The protocol reads every worker's configuration at each epoch boundary to set the next epoch's base fees, and the epoch-closing block writes each EIP-1559 worker's next-epoch base fee back to the contract. Governance can configure worker lane parameters without forking.
4. `Issuance`: Accepts TEL designated for validator rewards and uses the native ledger for accounting, providing verifiable reward distribution.

These contracts anchor the consensus protocol's economic and governance logic within the execution layer. The data provides verifiable committee transitions, staking state, and reward calculations without requiring trust.

Telcoin governance owns the `ConsensusRegistry` and `WorkerConfigs` through a multisignature Safe account. As owner, it approves validators, sets the stake versions and committee size, funds the `Issuance` contract, adjusts worker fee settings, and can pause validator and stake management in an emergency; the epoch-boundary system calls cannot be paused.

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

5. Epoch boundaries

Epochs are timestamp-based intervals whose duration is governed by the `epochDuration` parameter in the `ConsensusRegistry` contract. On the Adiri testnet, epochs last about six hours. Governance can change the duration from the governance Safe, a multisig, by publishing a new stake configuration, and each epoch keeps the duration in force when it opened. An epoch boundary is reached when a committed sub-DAG's commit timestamp, in whole seconds, reaches the epoch's end time: the epoch's start plus its duration. When this happens, a `close_epoch` flag triggers a deterministic sequence of system calls. All validators execute these calls identically, producing consistent state transitions across the network.

Four system calls run in strict order in the epoch's closing block: `applyIncentives()`, `applySlashes()`, and `concludeEpoch()` on the `ConsensusRegistry`, then `setWorkerConfigsData()` on the `WorkerConfigs` contract, which records the next epoch's base fees.

Reward distribution

Throughout the epoch, the protocol maintains a count of consensus headers for every validator. This count specifically reflects how often a commit was initiated by that validator's leader certificate. Leaders whose rounds produce empty commits still receive credit. This accounting is maintained in memory during normal operation and rebuilt from committed blocks if a node restarts as part of the recovery process.

When an epoch concludes, the protocol initiates the `applyIncentives()` function on the `ConsensusRegistry`. This function is supplied with each validator's address and their respective consensus header count. The `ConsensusRegistry` weights each validator's share of the epoch's issuance by its stake and its header count, and credits that share to the validator's reward balance. The closing block's `withdrawals` field, a repurposed Ethereum withdrawal list, records each validator's address and header count rather than a TEL amount, so explorers can see the reward inputs without tracing system calls. Validators claim earned rewards through a pull-only flow, so individual claim failures cannot revert the consensus-critical system call. During the subsidized growth phase, rewards are funded by the governance Safe.

Slash application

After distributing rewards, the protocol invokes `applySlashes()` on the `ConsensusRegistry`. Slashing is not applied: the protocol supplies an empty slash list, so the call leaves every balance unchanged.

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

Committee shuffle

After slashes are applied, each node draws the new committee itself and passes it to `concludeEpoch()` on the `ConsensusRegistry`, which validates it, settles queued stake changes, and records the committee.

The protocol reads the committee-eligible validators from the registry: those with `Active` or `PendingActivation` status, with `PendingExit` validators held in reserve. If `Active` and `PendingActivation` validators cannot fill the target committee size, `PendingExit` validators drawn at random fill the remaining seats.

Randomness comes from a 32-byte epoch seed chain. Each epoch starts from a root derived from the epoch number, and every commit folds in the committing leader's round and its deterministic BLS seed signature over its epoch, its round and the previous epoch record's digest, which the leader's header carries and its certificate covers. The value as of the epoch's closing commit seeds the shuffle and is published in the closing block's `extra_data`. The seed is deterministic, since every node folds the same signed inputs in the same order, and no one can compute it before the preceding commit is published. No proposer can grind the seed by re-cutting a proposal. The one residual influence belongs to the closing leader, which can compute the seed before broadcasting and withhold its proposal at the cost of that commit; the design accepts this bias.

The eligible validator set is shuffled using a Fisher-Yates algorithm seeded with the BLS-derived 32-byte seed. The shuffled list is truncated to the target committee size, which governance sets through the registry's `nextCommitteeSize` parameter. The selected members are then sorted by address for deterministic indexing; `concludeEpoch()` rejects a committee that is unsorted or the wrong size. The registry records the new committee three epochs ahead: the committee drawn when epoch N closes serves epoch N+3, so throughout any epoch the committees of the next two epochs are already fixed.

Base fee adjustment

Each epoch boundary also adjusts base fees using the worker's on-chain configuration. A gas accumulator tracks aggregate block count, gas used, and gas limit for each worker lane over the course of the epoch. From those totals and each worker's fee strategy, the closing block computes every worker's base fee for the next epoch and records each EIP-1559 worker's fee in the `WorkerConfigs` contract, which already holds each static worker's fee in its configuration. Every node entering the epoch, including one that restarts or syncs, reads its fees from the contract, and each batch must carry exactly that fee.

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

Each worker follows one of two governance-set strategies: EIP-1559 adjustment toward a gas target, or a fixed fee. The genesis configuration uses EIP-1559 with an effectively unlimited gas target, so fees stay at the minimum floor until governance sets a finite target. When gas usage equals the target, the base fee remains unchanged; when usage exceeds the target, the fee increases; when usage falls below the target, the fee decreases, subject to a minimum floor of 7 wei. After adjustment, all gas accumulators are cleared for the next epoch.

Fork scheduling

Protocol forks are scheduled at epoch boundaries. Each fork that would change the result of already-produced history activates at a fixed epoch number, so it takes effect exactly at an epoch boundary, where a new committee takes over and the closing block has settled the previous epoch.

Fork gates

Each such fork is gated on an epoch number compiled into the node. A gate compares its fork epoch with the epoch carried inside the data it handles, such as a header's own epoch or the committing leader's epoch, never with the node's local view, so replayed history reproduces exactly and every value from the fork epoch onward follows the new rule. Most gates stay in force for every epoch at or after their fork epoch; one-time contract upgrades instead run once, inside the closing block of the epoch before. Because the schedule is compiled into the node, a node whose release lacks a gate falls out of step with the network at, or soon after, that gate's fork epoch. The Adiri testnet carries history from before these forks, so each gate has its own activation epoch there. A network whose genesis postdates a fork applies that rule from its first epoch. The gate for millisecond-precision consensus timestamps has no activation epoch on Adiri, where commit times stay in whole seconds.

The gates armed on Adiri activate at these epochs:

Fork	Activation epoch (Adiri)
Seed signatures: every header carries its author's BLS seed signature, and the epoch seed chain replaces the leader-certificate seed for the committee shuffle	383
Registry upgrade: the `ConsensusRegistry` and `WorkerConfigs` contracts are upgraded in place, giving the registry per-status validator sets for the committee draw; from then on the closing block also calls `applySlashes()` and records next-epoch base fees	407

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

Fork	Activation epoch (Adiri)
Leader-seeded ordering: certificates committed in the same round are ordered by a key derived from the seed chain, so no proposer can steer its place in the commit order	567
Multiple workers: the committee description stored with each epoch's consensus data can list more than one worker per validator, lifting the single-worker limit	570
Seed-chain `PREVRANDAO`: each block's `PREVRANDAO` value comes from the seed chain instead of the consensus output and batch digests, so a leader cannot choose it by re-cutting its payload	574

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

6. Validator lifecycle

Validators must hold a ConsensusNFT issued by governance and stake the exact amount of TEL set by the current stake version in the `ConsensusRegistry` contract in order to participate in consensus. The stake requirement is based on the stake configuration when the node joins. Governance may later publish a new stake version that raises or lowers the required stake; a new version applies from the next epoch. Validators are not obligated to adjust their stake if the configuration changes. A validator, or its delegator, may opt in to a newer stake version; the change is one-way, and for a validator in service it settles at an epoch boundary, with any extra stake paid up front and any surplus refunded. Each epoch's issuance is shared in proportion to a weight equal to the validator's stake amount multiplied by the number of committed consensus rounds it led in that epoch. Validators on the same stake version therefore earn by performance alone, and where stake amounts differ, rewards are also weighted by the amount staked.

Telcoin Network uses a membership model rather than stake-weighted consensus. Each validator stakes exactly the amount of its stake version, no more; every committee member has equal voting power, and committees are drawn uniformly at random from the `Active` and `PendingActivation` validators, with `PendingExit` validators used only to fill a shortfall. An operator cannot add capital to one node to increase its influence, so it grows its share by running more governance-approved validators, each with its own ConsensusNFT, keys and stake.

ConsensusNFT

Governance mints a non-transferable ConsensusNFT (ERC-721) to a validator's execution address. Only the contract owner, managed by Telcoin governance, may mint these NFTs. The token ID is the validator's unique identifier. ConsensusNFTs are non-transferable and are burned upon unstaking.

Governance can also burn a validator's ConsensusNFT to eject it, unless that would leave a committee empty. The registry removes the validator from the current committee and the two committees already drawn after it, moves its staked TEL to the `Issuance` contract for future rewards, and retires its address; any unclaimed rewards are forfeited. The ejected validator stops voting no later than the next epoch boundary. The registry also supports slashing at epoch boundaries, with a validator slashed to zero ejected the same way, but the protocol does not submit any slashes.

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

Validator statuses

Each validator occupies exactly one status at any given time:

Status	Meaning
`Undefined`	The address has not staked. It may already hold a ConsensusNFT.
`Staked`	Validator has staked TEL but has not yet activated
`PendingActivation`	Validator is queued for activation; transitions to `Active` at the next epoch boundary
`Active`	Validator is eligible for committee selection indefinitely
`PendingExit`	Validator has initiated exit. It keeps the committee seats already assigned to it and is drawn into new committees only when `Active` and `PendingActivation` validators cannot fill them.
`Exited`	Validator is removed from committees and awaiting the cooldown period to unstake
`Retired`	The validator has unstaked or been ejected. Its ConsensusNFT is burned and the address can never rejoin. The registry marks retirement with a flag rather than a status value of its own.

Lifecycle steps

1. **NFT mint.** Telcoin governance mints a ConsensusNFT to the validator's execution address, granting that address the right to participate in the protocol.
2. **Key generation.** The validator generates a BLS12-381 keypair for consensus signing and Ed25519 network keys, derived from the BLS key, for peer identity and transport. It also signs a proof of possession that binds the BLS key to its execution address.
3. **Stake.** The validator calls `stake()` on the `ConsensusRegistry` with its BLS public key and proof of possession, sending exactly the stake amount of the current epoch's stake version. The registry verifies the proof through the BLS precompile and accepts each BLS key only once. The validator transitions from `Undefined` to `Staked`. Stake amounts are configurable to support iterative adjustment during early protocol phases.
4. **Activate.** The validator calls `activate()` to enter the `PendingActivation` queue. The designated activation epoch is the next epoch.

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

5. **Committee entry.** At the next epoch boundary, the `concludeEpoch()` system call moves the validator from `PendingActivation` to `Active`. The validator is already eligible for the committee drawn at that boundary. Committees are drawn three epochs ahead, so the earliest committee the validator can join serves from the third epoch after that boundary.

6. **Exit.** The validator calls `beginExit()` and enters `PendingExit` status. While in `PendingExit`, the validator keeps the committee seats already assigned to it and is drawn into new committees only if there are not enough `Active` and `PendingActivation` validators to fill them. The protocol moves the validator to `Exited` at the first epoch boundary at which it holds no seat in the committee of the starting epoch or in either of the two committees already drawn after it. With enough active validators this happens within three epoch boundaries of `beginExit()`.

7. **Unstake.** After one full epoch in the `Exited` state, the validator or its delegator calls `unstake()`. The registry returns the stake and any accrued rewards to the stake originator: the delegator when there is one, otherwise the validator. The ConsensusNFT is burned. The validator's address enters permanent retirement and can never rejoin the protocol. To participate again, the operator must obtain a new ConsensusNFT for a new address and generate a new BLS key, because a registered BLS key can never be registered again.

A validator that has staked but not yet called `activate()` can unstake at any time. A validator can claim accrued rewards without exiting once they reach its stake version's minimum withdrawal amount, and rewards always go to the stake originator.

Delegation

The protocol supports delegated staking, though it is expected to be used sparingly given the small, governance-vetted validator set. A delegator funds one validator's entire stake on its behalf; there is no pooling of stake from several delegators and no added consensus weight.

A delegation is formed before the validator stakes, and the validator authorizes it by signing it, unless governance forms it directly. Once formed, the delegator cannot be changed, and either the validator or the delegator can claim rewards or unstake. Upon unstaking, delegators receive all stake rewards and the full staked balance. Any reward splitting between a validator and its delegator is agreed upon offchain and falls outside protocol enforcement.

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

7. Node modes

Every node in the Telcoin Network operates in exactly one of three modes at any given time. The mode determines which protocol responsibilities the node performs and how it receives consensus output.

CvvActive

A node operates in `CvvActive` (Committee-Voting Validator, Active) mode when it holds a staked authority identity that is a member of the current committee and is keeping up with the current consensus round. A committee member enters this mode at the start of each epoch without first checking that it is caught up, and drops to `CvvInactive` if it finds itself behind. It stays `CvvInactive` instead if it is still catching up from the previous epoch, and runs as an observer if the newest epoch record it holds already names a next committee without its key.

A `CvvActive` node participates directly in consensus by producing batches, voting on proposals, generating certificates, and advancing consensus rounds. It executes each block as the consensus protocol commits it. Because it is a direct participant in consensus, it does not consume consensus output streams from peers.

CvvInactive

A node operates in `CvvInactive` (Committee-Voting Validator, Inactive) mode when it holds a staked committee identity but has fallen behind the current consensus state: more rounds behind than the consensus garbage-collection window allows, or an epoch behind. This can follow a restart mid-epoch.

A `CvvInactive` node cannot produce certificates, propose batches, or vote in consensus rounds. It follows consensus output by streaming ConsensusHeaders from peers, executes committed blocks as they arrive, and synchronizes its state forward. Once it has caught up, it rejoins consensus as `CvvActive` if its key is in the current committee, and runs as an observer otherwise.

Observer

A node operates in `Observer` mode when it is not a member of the current committee, or when the newest epoch record it holds names a next committee without its key. An observer does not vote in the current epoch, regardless of whether it holds a stake. This includes staked validators not selected for the committee in the current epoch and any node whose key is not in it. Each node derives its role from committee membership every epoch.

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

An observer follows consensus output by streaming ConsensusHeaders from peers and executes committed blocks as they arrive. It synchronizes the epoch chain but does not participate in consensus rounds, certificate production, or batch proposal. An observer does not put transactions into batches. Its workers forward the transactions they accept to committee validators over RPC (see Transaction forwarding); an observer that still holds a seat in the current committee keeps them in its pool instead. It does not subscribe to the committee's batch gossip, because it receives batch contents inside the consensus output it downloads.

Mode detection

At each epoch boundary, a node determines its mode for the new epoch by reading the current committee from the execution state. If the node's key is in the new committee, it enters `CvvActive`, unless the newest epoch record it holds names a next committee without that key, in which case it runs as an observer. If the key is not in the committee, the node enters `Observer` mode. A node that is `CvvInactive` at the epoch boundary stays `CvvInactive` until it has caught up. It then checks the committee again: it becomes `CvvActive` if its key is in the current committee and an observer if not. Nodes outside an epoch's committee run as observers for that epoch, once any catch-up carried over from the previous epoch has finished.

Mode transitions may also occur within an epoch. A `CvvActive` node that falls behind transitions to `CvvInactive`. A `CvvInactive` node that finishes catching up transitions to `CvvActive`, or to `Observer` if it no longer holds a seat.

Capabilities

Capability	`CvvActive`	`CvvInactive`	`Observer`
Consensus voting	Yes	No	No
Certificate production	Yes	No	No
Batch production	Yes	No	No
Transaction intake	Seals into batches	Holds in its pool until active	Forwards to committee validators
Block execution	Yes	Yes	Yes

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

Capability	`CvvActive`	`CvvInactive`	`Observer`
Consensus output streaming	No (direct participant)	Yes	Yes
Epoch chain sync	Yes	Yes	Yes

Transaction intake follows the committee seat rather than the mode: a node outside the current committee forwards transactions, and a committee member that is not `CvvActive` holds them in its pool.

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

8. Networking

Telcoin Network nodes communicate over a structured peer-to-peer overlay that combines publish-subscribe gossip, direct request-response messaging, bulk sync streams, and DHT-based discovery. All peer-to-peer communication uses QUIC v1 over UDP; transaction forwarding instead uses the JSON-RPC endpoints that validators advertise.

Transport

Every peer-to-peer connection requires a static IPv4 or IPv6 address. Node addresses follow the multiaddr convention:

```
/ip4/{HOST}/udp/{PORT}/quic-v1
```

Connections maintain periodic keep-alive probes to prevent NAT and firewall timeouts. A connection attempt, handshake included, is abandoned after about ten seconds, and a connection whose peer stops answering is closed by the QUIC idle timeout within about half a minute.

Identity and discovery

Each node holds a BLS12-381 keypair, and its network identity is an Ed25519 keypair derived from that BLS key (see Ed25519 signatures). The primary and each worker run their own swarm with their own derived network key, so a node has one peer ID per swarm; the BLS public key is the identifier that ties them together.

The BLS key signs the node's consensus messages when it sits on the committee, and it is the key registered on-chain. To enable discovery, every node publishes a node record to the Kademlia DHT of each of its swarms, signed with its BLS key, and republishes it periodically. Each record is keyed by the node's primary BLS public key and carries the swarm's Ed25519 network public key (from which the peer ID follows), its advertised addresses, a publication timestamp and, on worker records of validators that opt in, a public RPC endpoint. Any node can therefore resolve a validator's connection details from its BLS public key alone. The BLS signature covers the chain id and the role (primary or worker id), so a record verifies only on the network it was signed for.

The DHT uses standard Kademlia routing with a K-bucket size of 20. Bucket inserts are performed manually with record filtering to prevent pollution. Nodes target more connected peers than the K-bucket size to maintain routing table coverage.

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

Gossipsub topics

Message dissemination uses the gossipsub publish-subscribe protocol, negotiated under the chain-specific protocol ids `/tn-meshsub-{chain_id}/1.1.0` and `/tn-meshsub-{chain_id}/1.0.0`, across four topics. Every topic name carries the genesis chain id (2017 on the Adiri testnet), and the worker topic also carries the worker id, so nodes on different chains never share a mesh. Each topic accepts messages only from an allowlist of BLS keys that the node refreshes at the start of every epoch:

Topic	Authorized publishers	Purpose
`tn-primary-{chain_id}`	Current committee	Certificate propagation between committee members
`tn-consensus-output-{chain_id}`	Previous, current and next committee	Signed announcements of each committed consensus header (epoch, round, number and hash); a header signed by one-third plus one of the committee is trusted by observers and syncing nodes
`tn-epoch-vote-{chain_id}`	Previous, current and next committee	Epoch boundary vote signatures
`tn-worker-{chain_id}-{worker_id}`	Current committee	Batch digest announcements for one worker; only committee members subscribe

Each topic enforces message validation rules specific to its payload type. Messages that fail validation result in peer score penalties that may result in a node temporarily banning a peer.

Transaction forwarding

Transactions are not gossiped. A node outside the current committee accepts a transaction into its own pool and forwards it with `eth_sendRawTransaction` to a committee validator, at the JSON-RPC endpoint that validator advertises in its BLS-signed worker node record. The node takes the committee for the epoch from chain state and sends each transaction first to the validator whose committee slot owns the sender's address, so every transaction from one account reaches the same validator and nonce order is preserved. If that validator advertises no endpoint, cannot be reached or refuses the transaction for a reason local to its own node, the node tries the other committee validators in a rotating order, and a rejection of the transaction

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

itself stands only once a second validator confirms it, or when no other validator can be reached to consult. A transaction that does not reach the committee stays in the node's pool and is sent again.

Request-response RPC

Point-to-point communication uses a request-response protocol whose identifier names the node's role and the chain id, so a primary only negotiates with primaries, each worker only with workers of the same index, and nodes on different chains never negotiate at all:

```
/tn-primary-{chain_id}/0.0.2
/tn-worker-{worker_id}-{chain_id}/0.0.2
```

This protocol supports direct peer messaging for operations that require targeted delivery rather than broadcast:

- Header votes: a primary sends its proposed header, with any parent certificates the peer may lack, and receives a vote or a list of the parent certificates the peer still needs
- Batch reports: a worker sends a sealed batch to its peers, which accept or reject it
- Epoch records: a node requests an epoch record, together with the epoch certificate that signs it, by epoch number or by digest, or asks for the latest record the peer holds

Bulk transfers do not use request-response. Missing certificates, epoch consensus data, a single consensus output by its consensus number, and worker batches by digest travel over a separate sync-stream protocol, where one request opens a stream that the responder fills and then closes.

Each protocol interaction enforces a maximum message size to bound resource consumption. A message whose declared length exceeds the limit is rejected from its length prefix, before the node allocates memory for it, and the peer that sent it is penalized.

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

Peer scoring

Nodes keep a reputation score for every peer they know, connected or not, as a rational number in the range [-100, 100]. A new peer starts at 0. Reflecting the diminished relevance of older behavior, scores are designed to naturally decay toward zero over time using a configurable half-life. The thresholds and durations in this section are protocol defaults and may change between releases.

Penalties reduce a peer's score for specific infractions:

- Propagating invalid or malformed messages
- Failing message validation on gossipsub topics
- Violating protocol expectations

Three reputation levels derive from the current score:

Level	Score	Behavior
`Trusted`	above -20	Full participation. When a node holds more peers than its target, it drops the lowest-scoring peers first.
`Disconnected`	-20 or below	Connection closed; reconnection refused for ten minutes
`Banned`	-50 or below	Disconnected, blocked from gossip, removed from the discovery routing table, and refused new connections until the score recovers

A peer whose score falls to -50 or below is banned. A banned peer's score is frozen for 30 minutes before decay begins, which prevents peers from evading penalties through rapid reconnection. A ban therefore lasts at least that long, unless the peer joins a tracked committee, as described below. The peer is refused new connections throughout and is unbanned once decay lifts its score above -50.

Members of the previous, current and next committees are exempt from scoring: penalties and bans do not apply to them, and they may connect beyond the normal peer limits. When a validator joins a tracked committee, any ban on it is lifted and its score is reset to the maximum.

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

9. Cryptographic primitives

Telcoin Network uses BLS12-381 signatures for consensus, Ed25519 signatures for peer-to-peer network identity, BLAKE3 for consensus-layer digests and Keccak-256 at the execution layer. Transactions carry Ethereum's standard secp256k1 signatures. An intent-scoped signing scheme prevents cross-context signature replay.

BLS12-381 signatures

All consensus-layer signing uses the BLS12-381 curve. Validators sign votes on proposed headers, which aggregate into certificates; epoch votes on each epoch record, which aggregate into epoch certificates; attestations to committed consensus output; and a per-round seed signature carried in every header they propose. Every node, validator or not, also signs the node records it publishes for peer discovery. Multiple validator signatures aggregate into a single compact signature, so a verifier performs one aggregated signature check against the known committee public keys rather than N individual checks. Telcoin Network uses the minimal-signature-size variant: signatures are 48-byte points on G1 and public keys are 96-byte compressed points on G2. Messages are hashed to the curve with this domain separation tag:

```
BLS_SIG_BLS12381G1_XMD:SHA-256_SSWU_RO_NUL_
```

Validators must submit a proof-of-possession during key registration to prevent rogue key attacks. This proof demonstrates control of the private key corresponding to a registered public key before that key participates in any aggregation. The proof of possession is a BLS signature over a 119-byte message: the 3-byte proof-of-possession intent prefix (`0x000000`), the validator's 96-byte compressed public key and its 20-byte execution address, so the proof also binds the consensus key to the staking address. The `ConsensusRegistry` contract rebuilds this message on chain, verifies the signature through the BLS precompile and refuses a public key that is already registered.

Contracts can verify BLS signatures by calling a native precompile:

```
address    0x000000000000000000000000000000000000b151
function   blsVerify(bytes signature, bytes pubkey, bytes message)
```

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

The precompile runs the same verification code the consensus layer uses and treats the message as opaque, so any contract can check any BLS-signed message of up to 4,096 bytes; the `ConsensusRegistry` uses it for proofs of possession. A call costs 150,000 gas plus 12 gas per 32-byte word of arguments, and a call that carries value is rejected.

Ed25519 signatures

Ed25519 is the signature scheme for network-layer identity. Each node holds Ed25519 keypairs, one for its primary network and one for each worker network, derived deterministically from its BLS key so they never need to be stored. The public key determines the node's libp2p peer ID. The key authenticates the node's QUIC connections and signs every gossip message, and peers reject unsigned gossip. Node records published to the DHT are signed with the node's BLS key instead, so a record can be checked against the validator's registered consensus key.

Hashing

The consensus layer hashes with BLAKE3: batch, header, ConsensusHeader and EpochRecord digests are 32-byte BLAKE3 hashes, and a certificate is identified by its header's digest. The execution layer uses Keccak-256, which keeps it compatible with Ethereum: block hashes, transaction hashes and account addresses follow Ethereum's conventions.

Keccak-256 also builds the epoch seed chain, which folds each committed leader's BLS seed signature into the seed for the committee shuffle (see Committee shuffle).

Intent-scoped signing

Every consensus BLS signature commits to an IntentMessage that binds it to a specific protocol context. The message is signed after a 3-byte intent prefix is added. This prefix is composed of the scope, version, and application identifier.

IntentScope values define the signing context:

Scope	Value	Purpose
`ProofOfPossession`	0	Proof of possession for protocol keys
`EpochBoundary`	1	Authority signature on epoch records
`ConsensusDigest`	2	Votes on proposed headers (aggregated into certificates) and attestations to committed consensus output

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

Scope	Value	Purpose
`SystemMessage`	3	Reserved; no protocol message is signed under it
`EpochCloseSeed`	4	Proposer's per-round seed signature, folded into the committee-shuffle seed

IntentVersion is currently `V0`, a version byte reserved for future compatibility.

AppId identifies the application domain:

Application	Value	Purpose
`Telcoin`	0	General Telcoin Network operations
`Consensus`	1	Consensus-specific operations

This scheme ensures that a certificate signature cannot be replayed as an epoch vote, and that identical serialized payloads produce distinct signed messages whenever the intent differs.

Signature verification

Certificate verification follows two paths. Direct verification checks an aggregated signature against the current committee's BLS public keys. Indirect verification applies to certificates fetched in bulk during catch-up. A fetched certificate that another certificate in the same set names as a parent is accepted through that reference, because the child's header digest commits to its parents' digests, and every chain of such references ends at a certificate whose signatures are checked directly.

A directly verified certificate needs signatures from a super-quorum of the committee, at least 2/3 + 1 of its members (two-thirds rounded down, plus one); an indirectly verified certificate inherits that guarantee through its digest link. Epoch certificates use the same threshold. This bound derives from standard BFT assumptions, guaranteeing that any valid quorum contains at least one honest signer.

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

10. Trustless synchronization

A node must verify the entire chain history independently, without relying on any peer, whether it is joining the network initially or rejoining after a period of downtime. The well-known genesis block is the sole trust anchor for syncing historic data. From this starting point, any node derives and verifies every subsequent committee and every consensus decision, and, unless it starts from a snapshot, every state transition.

A node may instead start from an exported state snapshot of a recent epoch. The snapshot keeps the genesis committee as the only trust anchor, but replaces re-execution of the history below the snapshot with a certified checkpoint. The node re-verifies the snapshot's chain of certified epoch records from the genesis committee, checks the epoch's consensus pack against the certified `final_consensus`, and accepts the execution state only if it matches the block that the certified `final_state` names. It then syncs forward from that epoch, and holds no historical state below the snapshot block.

Synchronization proceeds in three steps.

Step 1: Sync the epoch chain

The node downloads epoch records sequentially starting from epoch 0. For each record, it verifies four properties:

1. Position: the record is for the epoch that was requested, so a peer cannot answer one slot with a self-consistent record for another
2. Chain integrity: the record's `parent_hash` matches the previous record's cryptographic digest
3. Committee continuity: the record's `committee` equals the previous record's `next_committee`, or is the smaller committee that a governance ejection allows (see Epoch chain)
4. Authenticity: the record's certificate carries a valid BLS aggregate signature from a super-quorum of the record's committee

The epoch 0 record is verified against the genesis committee. Each subsequent epoch's expected committee is learned from the previous record's `next_committee` field. After processing all available records, the node has committee membership for every epoch and two checkpoint hashes per epoch: `final_consensus` (the last verified consensus header digest) and `final_state` (the corresponding execution state).

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

The newest record is certified only after the following epoch has begun: its outgoing committee signs it then, and the network does not wait for the certificate before it starts the new epoch. A syncing node accepts a record only with its certificate, so the newest checkpoint becomes available once that certificate has been gathered.

Because the network produces one epoch record per epoch, and an epoch lasts hours rather than seconds (six hours in the Adiri testnet genesis; governance can later change the epoch length), this step completes quickly even after years of operation.

Step 2: Sync consensus headers

For each completed epoch, the node downloads that epoch's consensus pack, its consensus headers together with their batches, from a peer. It checks each header's parent link forward from the previous record's `final_consensus` and keeps the pack only if its last header matches the epoch record's `final_consensus`. For the epoch in progress, it walks backwards from the latest verified hash, and when it is more than a few outputs behind it first downloads a verified prefix of that epoch's pack. On that backward walk the node requests each consensus output by its number, confirms that its header digest matches the verified hash, and takes the header's `parent_hash` as the verified hash for the next number down. This repeats until every consensus header of the epoch is downloaded. A header is trusted because its digest chains to a certified epoch record or to a committee-attested consensus hash. For each output that carries batches, the node also checks that every certificate author belongs to that epoch's committee. This step may proceed in parallel with epoch record sync for epochs that were already verified.

Verified hashes that seed the download come from two sources, in order of availability:

Source	When available	Notes
Epoch records (`final_consensus`)	After epoch record sync	One historical checkpoint per epoch
Live committee gossip	Once current committee is known	Latest consensus hash in real time, accepted once one-third plus one of the committee have each signed the same result

Each execution block also records, in `parent_beacon_block_root`, the digest of the consensus header that produced it. A restarted node uses it to find where execution stopped and to detect a fork, but it does not seed downloads.

Telcoin Association Confidential and Proprietary Information. 15 October 2026.

Step 3: Re-execute batches

Once consensus headers span a contiguous range, the node orders them forward. The protocol extracts the batches from each header's committed sub-DAG and executes one EVM block per batch in sequence. As it executes, the node checks each consensus output against its own chain: the output's leader names the execution block it was built on, and the node halts if that block is not in its executed chain. Each epoch's consensus pack also names, as its starting execution state, the previous epoch record's certified `final_state`, and a pack that names any other state is rejected. This step transforms a cryptographically verified transaction ordering into actual account state, balances, and contract storage.

Security properties

Four mechanisms secure the synchronization process:

- BLS signature verification: every epoch record carries an aggregate BLS certificate from a super-quorum of its committee, and the live consensus hash is accepted only once one-third plus one of the committee have each signed it; consensus headers, and the certificates inside them, are then trusted through hash chaining to one of these signed checkpoints
- Parent hash chaining: prevents reordering or omission of records in all three chains
- Committee continuity validation: each record's committee must equal the previous record's `next_committee`, or, after a governance ejection, be a subset of it with at least four members and at least two-thirds of the promised committee; the bound applies one epoch at a time, so successive ejections can shrink a committee further
- Super-quorum threshold (2/3 + 1): safety holds while fewer than one-third of a committee's validators are Byzantine

Recovery after restart

A node that restarts mid-epoch does not repeat full synchronization. It rebuilds the epoch's gas accounting, leader reward counts and leader schedule from its locally stored blocks and consensus data. It then replays any consensus output that was committed but not yet executed, and resumes in the mode its committee membership gives it: a committee member resumes as `CvvActive`, unless the epoch records it syncs already rotate it out, and drops to `CvvInactive` if it finds itself behind; a node outside the committee resumes as an observer.

Telcoin Association Confidential and Proprietary Information. 15 October 2026.